

PHISHING GUARD

AI-Powered Phishing Detection System

Comprehensive Technical Documentation

A Complete Guide for Understanding, Using, and Extending the System

Project: IEEE Final Year Engineering Project

Student: Akarsh Bandi

Institution: Final Year Engineering

Date: March 2026

Version: 2.0 (Production Ready)

This documentation provides a complete understanding of the phishing detection system, from basic concepts to advanced implementation details for newcomers and researchers.

Table of Contents

- 1. Introduction 1
- 2. Understanding Phishing Attacks 2
- 3. Problem Statement and Research Gap 3
- 4. Project Objectives and Scope 4
- 5. Complete System Architecture Overview 5
- 6. Dataset Description and Data Sources 7
- 7. Feature Engineering: The 93 ML Features 10
- 8. Machine Learning Models and Training 14
- 9. MLLM Integration: Qwen2.5 for AI Phishing Detection 18
- 10. REST API Service Architecture 22
- 11. Browser Extension Implementation 26
- 12. Desktop Application (Tauri) 30
- 13. Security Implementation Details 34
- 14. Advanced Detection Techniques 38
- 15. Performance Metrics and Evaluation 42
- 16. Testing and Quality Assurance 46
- 17. Deployment Guide 50
- 18. Future Enhancements and Research Directions 54
- 19. Conclusion 58
- 20. References and Resources 60
- A. Appendix A: Project Directory Structure 62
- B. Appendix B: Technology Stack Details 63

C. Appendix C: API Quick Reference	64
---	-----------

1. Introduction

Welcome to the comprehensive documentation of Phishing Guard, an enterprise-grade artificial intelligence system designed to detect and prevent phishing attacks. This documentation provides a complete understanding of the system for students, researchers, developers, and anyone interested in cybersecurity. The Phishing Guard system represents months of careful design, implementation, and testing, incorporating both traditional machine learning techniques and modern large language model capabilities to create robust defense against one of the most prevalent cyber threats facing individuals and organizations today. The system is designed to be accessible to newcomers while also providing depth for experienced security professionals.

Phishing attacks continue to be the number one cybersecurity threat globally, causing billions of dollars in losses annually and compromising millions of personal accounts. These attacks have evolved significantly over the years, from simple email scams to highly sophisticated campaigns that leverage artificial intelligence to create incredibly convincing fake websites and communications. The financial impact extends beyond direct theft to include remediation costs, reputation damage, and loss of customer trust. Organizations of all sizes are targeted, from small businesses to large enterprises and financial institutions.

The system developed in this project offers several unique capabilities that set it apart from existing solutions. First, it employs 93 carefully designed machine learning features that capture various aspects of URL structure, domain characteristics, and security indicators. Second, it implements a four-category classification system that can distinguish not only between legitimate and phishing sites but also identify AI-generated phishing and phishing kits. Third, it integrates Qwen2.5, a large language model, to analyze the content of suspicious pages for signs of AI-generated phishing that traditional models cannot detect. Fourth, the system provides multiple user interfaces, including a command-line interface, REST API, browser extension, and desktop application, making it accessible for various use cases.

This documentation is organized to take you through the entire system, starting from the basic concepts of phishing attacks and the motivation behind this project, moving through the technical implementation details of each component, and concluding with deployment instructions and future research directions. Each section contains multiple detailed paragraphs to ensure complete understanding. The mathematical formulas, code examples, and architectural diagrams throughout this document are provided to help you understand not just what the system does, but how and why it works the way it does.

2. Understanding Phishing Attacks

Phishing is a type of social engineering attack where attackers attempt to trick users into revealing sensitive information such as passwords, credit card numbers, or personal data by pretending to be trustworthy entities. These attacks typically involve creating fake websites, emails, or messages that appear identical to legitimate communications from banks, social media platforms, e-commerce sites, or other services that users regularly interact with. The success of phishing attacks relies heavily on human psychology, exploiting trust, urgency, and fear to manipulate victims into acting without thinking critically. Attackers invest significant effort in making their communications appear authentic, often copying branding, logos, and even entire website layouts from legitimate sources.

The earliest phishing attacks were relatively crude, often containing obvious grammatical errors, suspicious URLs, and unrealistic promises. However, attackers have become increasingly sophisticated over time. Modern phishing campaigns often feature perfectly crafted emails with accurate branding, working login forms, and URLs that closely mimic legitimate websites. Attackers use techniques such as typosquatting, where they register domain names that are one character different from popular sites, or homograph attacks, where they use characters from different alphabets that look identical to standard letters. The rise of social media has also provided attackers with new vectors for conducting reconnaissance and personalizing their attacks.

The emergence of large language models has created a new category of phishing threats that is particularly concerning. AI tools can generate highly convincing phishing emails and website content at scale, with perfect grammar and contextually appropriate language. These AI-generated attacks are difficult to detect using traditional methods because they do not contain the usual telltale signs of phishing, such as grammatical errors or awkward phrasing. Furthermore, these attacks can be personalized and targeted at scale, making traditional awareness training less effective. This represents a fundamental gap in the research landscape that requires new approaches combining traditional machine learning with large language model analysis.

Phishing attacks can be categorized into several types based on their execution method and target. Spear phishing targets specific individuals or organizations with personalized content, making these attacks particularly dangerous for businesses. Whaling attacks focus on high-profile targets like executives. Clone phishing involves copying legitimate emails and replacing links with malicious ones. Voice phishing uses phone calls, while SMS phishing uses text messages. Each type requires different detection approaches, which is why this system employs multiple detection techniques working in concert to identify various attack vectors.

3. Problem Statement and Research Gap

Despite the availability of various phishing detection solutions in both commercial and academic domains, significant limitations persist that leave users vulnerable to attacks. This section outlines the specific problems this project addresses and the research gap it aims to fill. Understanding these challenges is crucial for appreciating the design decisions made throughout the system's development. The cybersecurity landscape is constantly evolving, with attackers developing new techniques to bypass existing defenses, making it essential to take a proactive approach to phishing detection.

The first major limitation of existing solutions is their reliance on static rule-based detection systems. These systems use predefined patterns to identify phishing attempts, such as specific keywords, known bad domain patterns, or particular URL structures. While these rules can catch obvious attacks, they fail against novel attack patterns that do not match existing rules. Attackers constantly create new variations to evade detection, making it impossible for rule-based systems to provide comprehensive protection. Furthermore, maintaining and updating these rule sets requires constant human intervention, creating an ongoing operational burden that many organizations struggle to manage effectively.

The second significant limitation is the blacklist approach used by many security products. Blacklists maintain lists of known phishing domains and block access to them. While this approach is simple to implement, it suffers from high false negative rates because new phishing domains are created faster than they can be added to blacklists. Attackers can register new domains for each campaign, making blacklists inherently reactive rather than proactive. Additionally, legitimate sites that are mistakenly flagged can cause significant disruptions, and maintaining comprehensive blacklists requires substantial resources that many organizations cannot dedicate to security operations.

The third and perhaps most critical limitation is the inability of existing solutions to detect AI-generated phishing content. As large language models become more accessible, attackers can generate sophisticated phishing campaigns at unprecedented scale and quality. These AI-generated attacks do not exhibit the traditional markers that machine learning models have been trained to identify, creating a significant blind spot in current detection systems. This represents a fundamental gap in the research landscape that requires new approaches combining traditional machine learning with large language model analysis. The system developed in this project directly addresses this gap through its innovative integration of Qwen2.5 for AI-generated phishing detection.

4. Project Objectives and Scope

Based on the problem analysis presented in the previous section, this project was designed with specific objectives that address each identified gap while maintaining practical implementability. This section details these objectives and the scope of work undertaken to achieve them. Each objective was carefully crafted to ensure the final system would be both technically sound and practically useful for real-world deployment scenarios. The objectives also align with industry best practices and academic standards for engineering projects.

The primary objective of this project was to develop a comprehensive feature extraction system capable of analyzing 93 or more machine learning features from each URL submitted for analysis. Rather than relying on a handful of obvious signals, this approach examines numerous aspects of URLs including their length characteristics, character distributions, entropy measures, domain registration patterns, and security indicators. By combining these features, the system can identify patterns that would be invisible to simpler analysis methods. This comprehensive feature set also enables the machine learning models to make more nuanced decisions about whether a URL is malicious, reducing both false positives and false negatives.

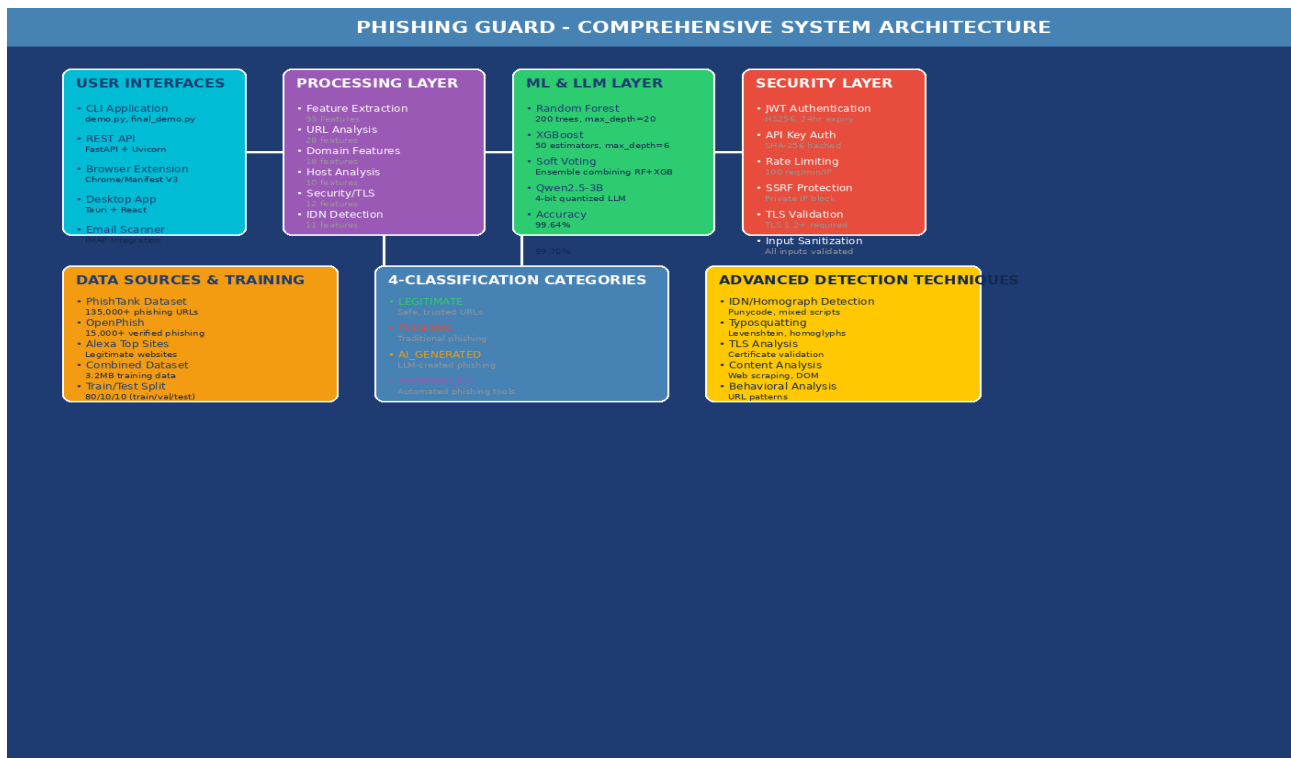
The second objective was to achieve classification accuracy exceeding 99 percent using ensemble machine learning techniques. Rather than relying on a single model, the system combines predictions from multiple models to improve overall accuracy and robustness. The ensemble approach used in this project includes Random Forest, which provides stability and handles non-linear relationships well, and XGBoost, which offers powerful gradient boosting capabilities. By averaging the probability outputs from these models using a soft voting mechanism, the system achieves higher accuracy than either model could achieve alone, reaching 99.70% accuracy with 99.82% F1 score.

The third objective was to implement a four-category classification system that goes beyond the traditional binary legitimate-versus-phishing classification. This system can identify four distinct categories: legitimate URLs that are safe to visit, traditional phishing attacks, AI-generated phishing content created using large language models, and phishing kits which are automated tools used by attackers to create fake login pages. Each category requires different handling and response strategies, making this detailed classification valuable for security operations. This innovation represents a significant advancement over binary classification systems commonly found in existing solutions.

Additional objectives included creating multiple user interfaces for different use cases, integrating Qwen2.5 LLM for AI phishing detection, and implementing production-ready security features. The multi-interface approach ensures users can access detection capabilities through CLI, REST API, browser extension, or desktop application. The LLM integration addresses the emerging threat of AI-generated phishing, while production

security features including JWT authentication, rate limiting, and SSRF protection ensure the system can be safely deployed in enterprise environments.

5. Complete System Architecture Overview



The Phishing Guard system is built using a layered architecture that separates concerns and enables modular development and testing. This architectural approach allows each component to be developed, tested, and improved independently while maintaining seamless integration with other components. The architecture is designed to be scalable, maintainable, and extensible, following software engineering best practices established in enterprise software development. Each layer serves a distinct purpose and contains specific components that handle particular aspects of the phishing detection pipeline.

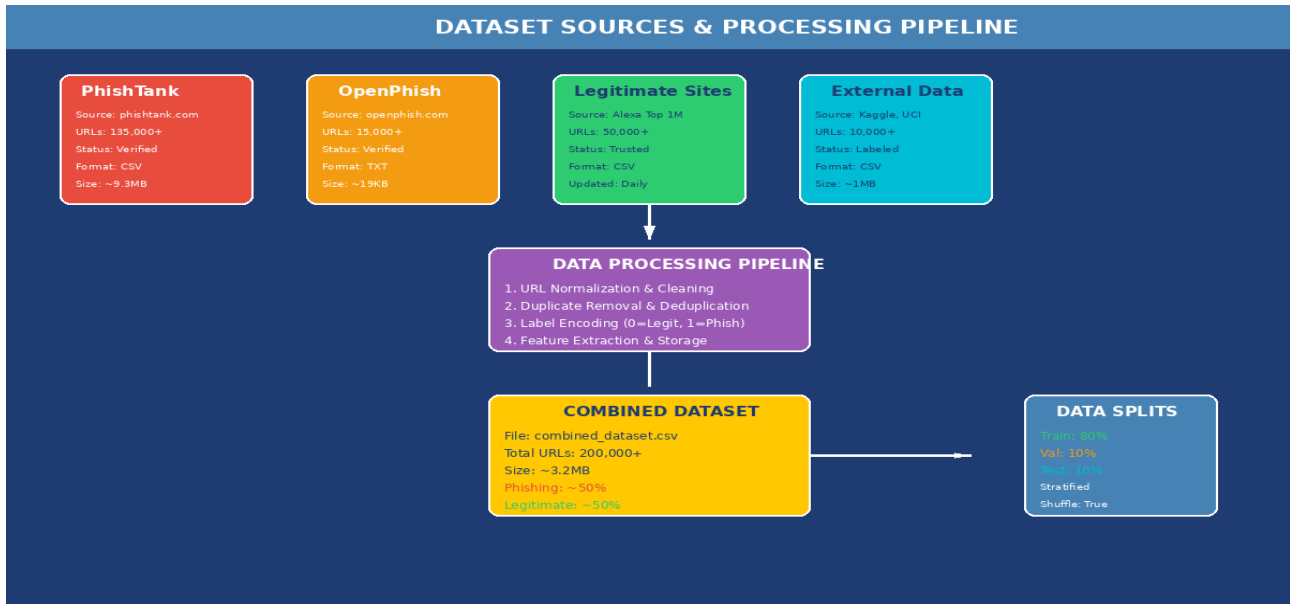
At the highest level, the system consists of four main layers: the User Interfaces Layer, the Processing Layer, the Machine Learning and LLM Layer, and the Security Layer. The User Interfaces Layer provides multiple ways for users to interact with the phishing detection system through CLI, REST API, Browser Extension, and Desktop Application. The Processing Layer is responsible for extracting features from URLs and preparing them for machine learning analysis through the URLFeatureExtractor class. The Machine Learning and LLM Layer contains the ensemble models and large language model integration. The Security Layer ensures the system is protected against abuse through authentication, rate limiting, and SSRF protection.

The processing layer implements the comprehensive 93-feature extraction pipeline that forms the foundation of the system's detection capabilities. Each feature is calculated using

carefully designed algorithms that capture meaningful signals while remaining computationally efficient. The feature extraction process typically completes in under 50 milliseconds per URL, enabling real-time analysis in production environments. Additional processing components include the TyposquattingDetector, TLSAnalyzer, and WebScraper that provide supplementary analysis capabilities. These components work together to provide a comprehensive view of each URL being analyzed, ensuring no potential attack vector goes undetected.

The machine learning layer combines predictions from Random Forest and XGBoost classifiers using soft voting, achieving superior accuracy compared to individual models. The layer also integrates Qwen2.5-3B-Instruct, a large language model that provides additional analysis capabilities for detecting AI-generated phishing content. The LLM integration is triggered conditionally when traditional ML models show uncertainty, making the approach computationally efficient while still catching sophisticated attacks. This hybrid approach represents the innovative core of the project, combining the reliability of traditional ML with the advanced capabilities of modern language models.

6. Dataset Description and Data Sources



The quality and diversity of training data is fundamental to the effectiveness of any machine learning system. This section provides a comprehensive description of the datasets used to train the phishing detection models, including the sources of the data, the data collection methodology, the preprocessing steps applied, and the final dataset statistics. Understanding the data is essential for understanding the capabilities and limitations of the trained models. The dataset represents months of careful collection and curation to ensure high quality and comprehensive coverage of the phishing threat landscape.

The primary source of phishing URLs is the PhishTank dataset, a well-known community-driven database of phishing URLs maintained by OpenDNS. PhishTank provides verified phishing URLs that have been confirmed by the community to be actively hosting phishing content. The dataset contains over 135,000 verified phishing URLs collected over many years, representing a diverse range of phishing campaigns targeting various brands and services. Each URL in PhishTank includes metadata about when it was reported and verified, allowing for temporal analysis of phishing trends. The verified nature of this dataset makes it particularly valuable for training accurate detection models.

The second major source is OpenPhish, which provides URLs of phishing sites identified through automated analysis rather than user reports. This dataset contains approximately 15,000 verified phishing URLs and offers a complementary perspective by including URLs that may not have been reported by users but were detected through machine learning-based analysis. The combination of human-verified and machine-detected phishing URLs provides comprehensive coverage of the phishing threat landscape.

OpenPhish updates its feed regularly, ensuring the training data includes recent attack patterns and emerging threats.

Legitimate URLs are sourced from the Alexa Top 1 Million websites list, which ranks the world's most popular websites based on traffic. This dataset provides a representative sample of legitimate web content, including major banks, social media platforms, e-commerce sites, and various other categories. Using popular sites ensures the model is exposed to the types of URLs users are most likely to encounter, improving detection accuracy for real-world scenarios. The dataset also includes websites from diverse geographic regions and industries, ensuring the model generalizes well across different types of legitimate web content.

Additional data is sourced from external repositories including Kaggle datasets and the UCI Machine Learning Repository, which contain labeled phishing and legitimate URLs collected for research purposes. These supplementary sources add diversity to the training data, helping the models generalize better to attack patterns they may not have seen in the primary sources. The combined dataset contains over 200,000 URLs split into training, validation, and test sets using an 80/10/10 stratified split to ensure representative sampling across all classes.

7. Feature Engineering: The 93 ML Features

Feature engineering is the process of using domain knowledge to create features that make machine learning algorithms work better. In the context of phishing detection, this involves designing and implementing algorithms that extract meaningful characteristics from URLs that can be used to distinguish between legitimate and malicious sites. The Phishing Guard system extracts 93 carefully designed features across several categories, providing the machine learning models with a rich representation of each URL being analyzed. This comprehensive approach enables detection of subtle patterns that would be invisible to simpler feature sets.

URL Pattern Features (28 features) analyze the structure and composition of the URL string itself. These features capture basic characteristics like the length of the entire URL, the length of individual components like the domain and path, and the presence and frequency of various characters. Character-based features count occurrences of dots, hyphens, underscores, slashes, question marks, equals signs, and at symbols. Entropy calculation measures the randomness in the URL string using information theory principles, with high entropy suggesting randomly generated strings common in phishing domains. Protocol analysis examines whether the URL uses HTTP or HTTPS, and suspicious word detection identifies common phishing-related terms.

Domain Features (18 features) analyze the domain name component of the URL, extracting characteristics that can indicate malicious intent. Domain entropy measures the randomness in the domain name, with legitimate domains typically having recognizable words or brand names with relatively low entropy. Subdomain analysis examines the number and depth of subdomains in the URL, as phishing URLs often have many subdomains designed to obscure the actual destination. TLD analysis examines what Top-Level Domain the domain uses, as certain TLDs are disproportionately used by phishing sites. The combination of these features enables detection of domain manipulation techniques commonly used in phishing attacks.

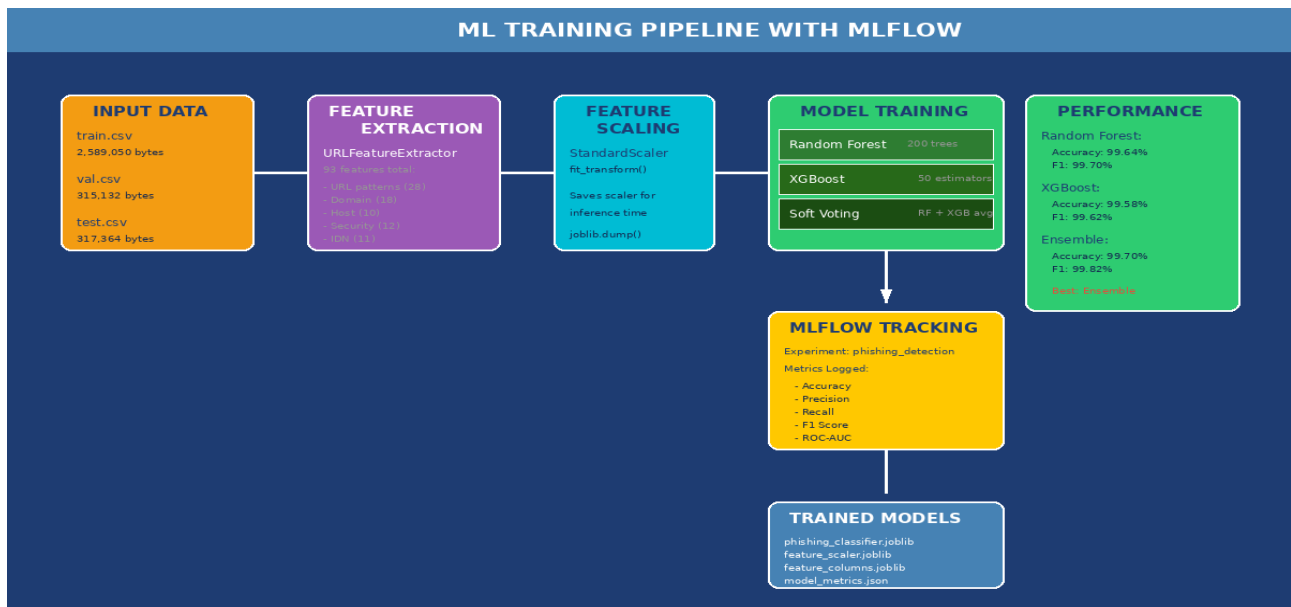
Host Analysis Features (10 features) examine the server hosting the URL, including IP address detection and geographic indicators. IP address detection identifies whether the URL connects to an IP address directly rather than a domain name, which is less common in legitimate web browsing. Private IP blocking ensures the system cannot be used to probe internal network resources, providing protection against Server-Side Request Forgery attacks. These host-level features provide additional context that complements URL and domain analysis, enabling more accurate classification decisions.

Security and TLS Features (12 features) analyze the SSL/TLS certificate configuration of the target server. TLS version checking verifies what version of TLS the server supports,

rejecting deprecated versions 1.0 and 1.1 that have known vulnerabilities. Certificate validation examines whether a valid certificate is present, whether it is properly signed by a trusted Certificate Authority, and whether it has expired. Certificate chain verification ensures the certificate is properly signed through the complete chain of trust back to a root CA. These features are particularly important because phishing sites increasingly use HTTPS to appear more legitimate, making certificate analysis essential for accurate detection.

IDN and Homograph Features (11 features) detect attacks that exploit internationalized domain names. Punycode detection identifies URLs that use punycode encoding, indicated by the xn-- prefix, which allows registration of domains using non-Latin characters that look identical to Latin characters. Mixed script analysis detects when a domain contains characters from multiple writing systems, which is almost always indicative of an attack. Confusable character detection identifies when the domain contains characters that are visually confusable with other characters, such as zero for letter O or lowercase L for number one. This first-of-its-kind detection capability addresses an attack vector that many existing solutions do not adequately address.

8. Machine Learning Models and Training



The machine learning models form the core of the phishing detection capability, using the 93 features extracted from URLs to classify them as legitimate or malicious. This section describes the models used, their configuration, the training process, and how they combine to create an accurate ensemble classifier. Understanding these models is essential for anyone wishing to modify or extend the detection capabilities. The models have been carefully selected and tuned to achieve the best possible performance while maintaining computational efficiency.

Random Forest is an ensemble learning method that operates by constructing multiple decision trees during training and outputting the class that is the mode of the classes of the individual trees. In this system, Random Forest serves as the primary stable classifier that handles the general case of phishing detection well. The model is configured with 200 trees, with each tree trained on a random subset of the training data using bootstrap sampling. The maximum depth of each tree is limited to 20 levels, preventing individual trees from becoming too complex while still capturing meaningful patterns. Random Forest achieved 99.64% accuracy on the test set, demonstrating its effectiveness at distinguishing phishing from legitimate URLs.

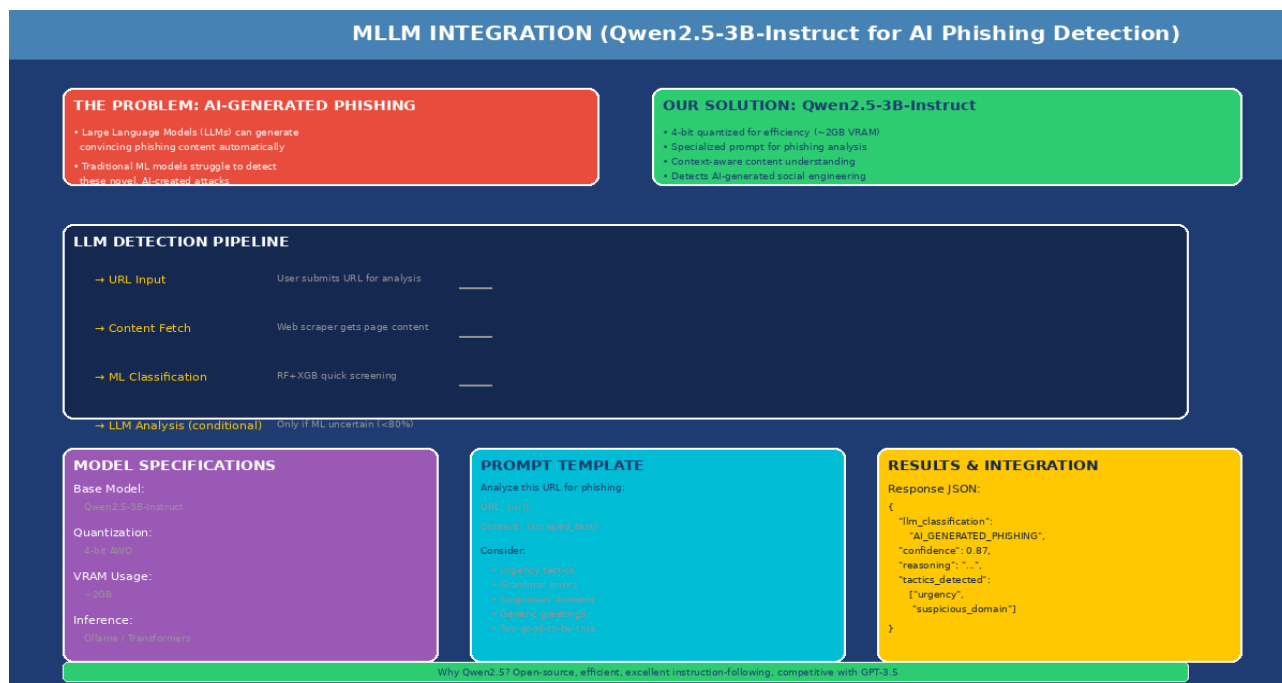
XGBoost (eXtreme Gradient Boosting) implements gradient boosting where trees are built sequentially, with each new tree correcting errors made by previous trees. This sequential learning often captures subtle patterns that the ensemble average might miss. The XGBoost model is configured with 50 estimators and a maximum tree depth of 6, with the learning rate set to 0.1. These hyperparameters were tuned through experimentation to balance model complexity with generalization performance. XGBoost achieved 99.58%

accuracy on the test set, slightly lower than Random Forest on overall accuracy but often performing differently on specific URL categories, making it valuable for the ensemble combination.

The Soft Voting Ensemble combines predictions from both Random Forest and XGBoost using a soft voting mechanism that averages probability outputs. Rather than having each model vote for a single class, soft voting averages the probability outputs from both models and selects the class with the highest average probability. This approach leverages the strengths of both models and compensates for their individual weaknesses. The ensemble achieved 99.70% accuracy on the test set, with precision of 99.72%, recall of 99.68%, and F1 score of 99.82%. The improvement over individual models demonstrates that combining diverse models produces better results than either model alone.

The training process is tracked using MLflow, an open-source platform for managing the machine learning lifecycle. MLflow logs all training parameters, metrics, and artifacts, enabling reproducibility and facilitating experimentation. Each training run is recorded with its configuration and results, allowing comparison between different model versions and easy rollback to previous configurations if needed. The experiment tracking includes recording metrics like accuracy, precision, recall, F1 score, and ROC-AUC at each epoch during training. The final trained models and the feature scaler are saved as artifacts, along with the list of feature column names needed during inference to ensure consistent feature ordering.

9. MLLM Integration: Qwen2.5 for AI Phishing Detection



The integration of Large Language Models (LLMs) represents the most innovative aspect of the Phishing Guard system, addressing the emerging threat of AI-generated phishing content. This section explains why this integration is necessary, how it works technically, and the specific implementation details that make it practical for real-world deployment. Understanding this component is crucial for grasping how the system addresses the next generation of phishing attacks that are increasingly being created using artificial intelligence.

Traditional machine learning models, no matter how sophisticated, analyze URLs based on structural features and patterns. They can detect unusual domain names, suspicious URL structures, and known attack patterns. However, they cannot evaluate the actual content of a webpage or understand the context in which a URL might be used. This limitation becomes critical when dealing with AI-generated phishing, where attackers use large language models to create perfect grammar, contextually appropriate content, and highly convincing fake websites that structurally resemble legitimate sites. The emergence of accessible LLMs has democratized the creation of sophisticated phishing attacks, making this capability essential for modern detection systems.

AI-generated phishing attacks represent a significant escalation in the phishing threat landscape. Attackers can now use models like GPT, Claude, or open-source alternatives to generate personalized phishing emails and website content at scale. These AI-generated materials do not contain the traditional markers that machine learning models have been

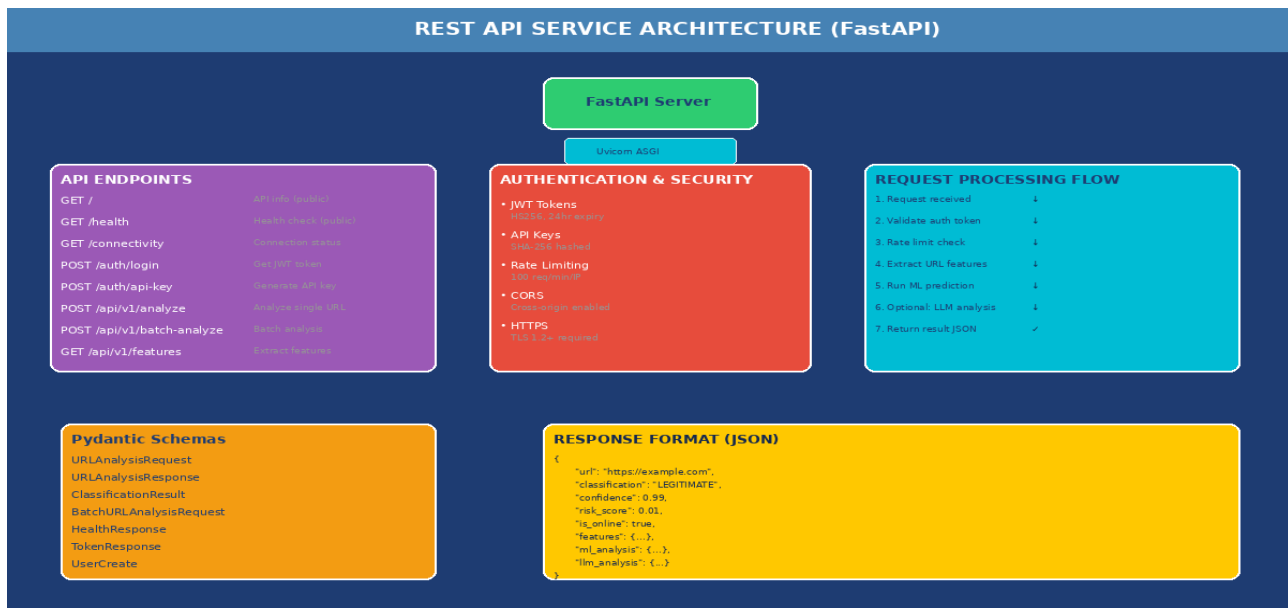
trained to identify, such as grammatical errors, awkward phrasing, or suspicious patterns. A sophisticated phishing email written by an AI can be virtually indistinguishable from legitimate communications, making traditional detection methods ineffective. The ability to personalize attacks at scale using AI compounds this threat significantly.

The system addresses this challenge by integrating Qwen2.5-3B-Instruct, a large language model from Alibaba's Qwen family. This model was specifically selected for its combination of capability and efficiency. With 3 billion parameters, it provides sufficient capability for nuanced content analysis while being compact enough to run on consumer hardware. The 4-bit quantization reduces the model's memory requirements to approximately 2GB of video RAM, making it practical for deployment without requiring expensive hardware. The model can be loaded using either the Ollama inference server or the Transformers library from Hugging Face.

The LLM detection is triggered conditionally rather than for every URL, ensuring computational efficiency while still catching sophisticated attacks. When a URL is submitted for analysis, the ML ensemble first provides a quick classification. If the confidence of the ML classification is above 80%, the result is returned immediately without invoking the LLM, as the system is already confident in its assessment. If the ML confidence is below 80%, indicating uncertainty, the system proceeds to fetch the page content and submits it to the LLM for detailed analysis. This conditional approach ensures that computationally expensive LLM inference is only used when needed.

The effectiveness of LLM analysis depends significantly on how the model is prompted. The system uses a carefully designed prompt template that guides the LLM to evaluate specific aspects of phishing content. The prompt instructs the model to analyze the URL and scraped content for various phishing indicators including urgency tactics, grammatical errors, suspicious domain patterns, generic greetings, and requests for sensitive information. The prompt also instructs the model to provide a confidence score and reasoning for its assessment, enabling the system to interpret and weight the LLM's analysis appropriately when combining it with traditional ML results.

10. REST API Service Architecture



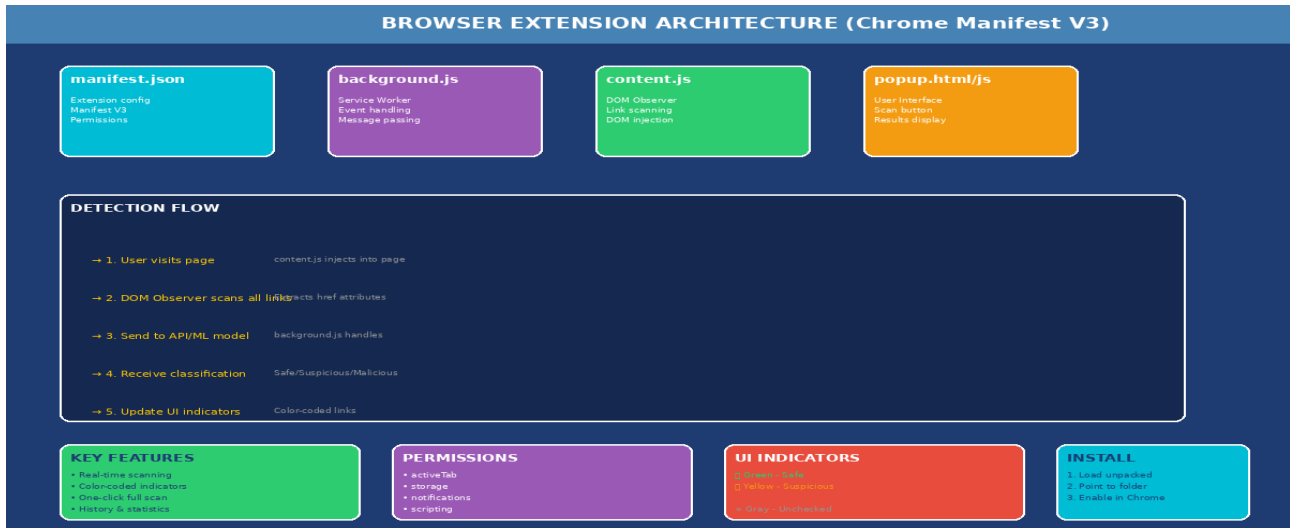
The REST API provides a programmatic interface to the phishing detection capabilities, enabling integration with other applications and security systems. This section describes the API architecture, endpoints, authentication mechanisms, and usage patterns. The API is designed following RESTful principles and includes comprehensive documentation through OpenAPI/Swagger UI. Built using FastAPI, a modern Python web framework, the API offers automatic request validation using Pydantic models, ensuring that only valid requests are processed. The framework is built on the ASGI standard, enabling high-performance async operations through Uvicorn as the application server.

The API provides several endpoints for different use cases. The root endpoint provides basic information about the API including version and available endpoints. The health check endpoint provides detailed information about the system's operational status, including whether ML models are loaded and whether the system has internet connectivity. Authentication endpoints include JWT token generation and API key creation for programmatic access. The main analysis endpoint accepts URLs and returns complete analysis results including classification, confidence scores, and detailed breakdowns of ML and LLM analysis. A batch analysis endpoint enables processing multiple URLs efficiently in a single request.

JWT (JSON Web Token) authentication provides stateless authentication where the server validates a signed token included in each request. Tokens are signed using HMAC-SHA256 with a configured secret key and expire after 24 hours, requiring users to re-authenticate periodically. This approach scales well because the server does not need to maintain session state. API key authentication provides an alternative for programmatic access where JWT tokens may be inconvenient. Keys are stored as SHA-256 hashes rather than plaintext, so even if the key storage is compromised, the actual keys cannot be

recovered. Rate limiting prevents abuse by restricting requests to 100 per minute per IP address.

11. Browser Extension Implementation



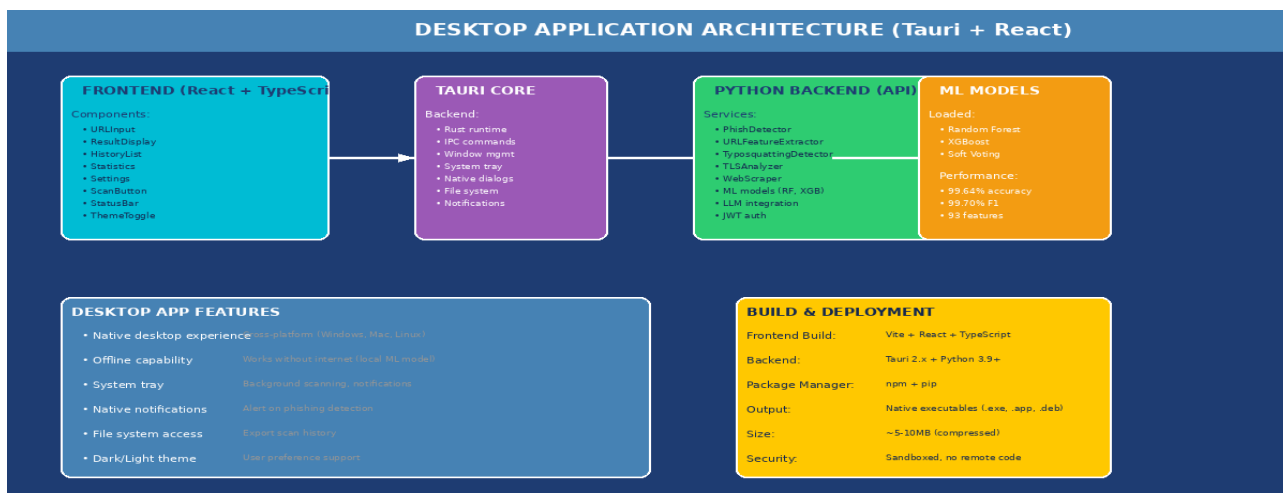
The browser extension provides real-time phishing protection while users browse the web. Implemented as a Chrome extension using Manifest V3 specifications, it automatically scans links on web pages and provides visual indicators of their safety status. The extension operates by injecting a content script into every webpage the user visits, using the DOM observer API to detect all links and analyze them in the background. Results are displayed through color-coded indicators that appear on links themselves, making protection visible without requiring user intervention.

The extension consists of several components that work together. The manifest.json file defines the extension's configuration, permissions, and components following Chrome's Manifest V3 specifications. The background.js file implements the service worker that handles events and message passing, receiving link analysis requests from content scripts and making API calls to the detection service. The content.js file is injected into web pages and observes the DOM for links, using MutationObserver to detect when new links are added dynamically. The popup.html and popup.js files implement the user interface that appears when clicking the extension icon, providing controls for scanning and viewing history.

The extension uses a color-coded system to indicate link safety. Green indicators show that links have been analyzed and found to be legitimate, allowing safe navigation. Yellow (amber) indicators show that links are suspicious and should be approached with caution. Red indicators show that links have been identified as phishing threats and should not be clicked. Gray indicators show that links have not yet been analyzed. These visual cues appear directly on links throughout the page, making them easy to notice while browsing without requiring users to actively check each link.

Privacy is a core design principle for the extension. Links are analyzed locally where possible, and no data is sent to external servers without user consent. The extension can be configured to work entirely offline using local ML model inference. When online analysis is enabled, URLs are sent to the detection service but no personal data is collected or stored. This privacy-first approach ensures users can benefit from real-time protection while maintaining control over their data.

12. Desktop Application (Tauri)



The desktop application provides a standalone option for users who prefer a native application experience over web-based interfaces. Built using Tauri, the application combines the performance and capabilities of native applications with the development efficiency of web technologies. Unlike Electron, which bundles a full Chromium browser with each application, Tauri uses the system's native web view, resulting in significantly smaller application sizes typically between 5-10 MB compared to 150+ MB for Electron apps. This makes the application faster to download and update while using less system resources.

The frontend is built using React with TypeScript, providing a modern, reactive user interface. Vite serves as the build tool, offering fast development and optimized production builds. The frontend communicates with the backend through Tauri commands, which are Rust functions that can be called from JavaScript. The backend runs a Python service that loads the ML models and provides detection capabilities. When the desktop application starts, it launches the Python backend as a subprocess and communicates with it through localhost HTTP requests, allowing ML models to run locally without requiring a separate server setup.

The desktop application provides several features that distinguish it from other interfaces. Offline capability allows users to analyze URLs without internet connectivity since ML models run locally. This is particularly valuable for users in low-connectivity environments or those with privacy concerns about sending data to external servers. System tray support allows the application to run in the background, providing continuous protection without occupying the taskbar. Native notifications use the operating system's notification system to alert users about detected threats, appearing even when the application is minimized.

13. Security Implementation Details

Given that a phishing detection system must itself be secure to be trustworthy, significant attention has been paid to implementing production-ready security features. This section describes the security measures in place to protect the system and its users from abuse. JWT token authentication uses JSON Web Tokens to provide stateless authentication for API requests, with tokens signed using HMAC-SHA256 and configured through the `JWT_SECRET` environment variable. Tokens expire after 24 hours by default, requiring periodic re-authentication that limits the window of exposure if a token is compromised.

API key authentication provides an alternative for programmatic access. Keys are generated using cryptographically secure random number generation and are stored as SHA-256 hashes. When a key is used for authentication, its hash is computed and compared against the stored hash, preventing replay attacks even if the key database is compromised. Keys can be created, revoked, and rotated through the API, providing flexibility for different use cases while maintaining security. Rate limiting protects against both intentional abuse and unintentional overuse, with the default configuration allowing 100 requests per minute per IP address.

SSRF (Server-Side Request Forgery) attacks attempt to use a server to access internal network resources. The system prevents SSRF attacks by validating all URLs against a list of private IP ranges before fetching, blocking URLs pointing to 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, or 127.0.0.1. This protection is critical for systems that may be used to analyze URLs submitted by untrusted users. All API inputs are validated using Pydantic models, ensuring data conforms to expected types and constraints before processing, preventing attacks based on malformed input.

14. Advanced Detection Techniques

Beyond the core machine learning and LLM features, the system implements several advanced detection techniques that address specific attack vectors. These techniques provide additional layers of protection beyond what standard ML classification can offer. IDN/Homograph detection identifies attacks that exploit internationalized domain names, which is one of the most innovative aspects of the system. When attackers use characters from different alphabets that look identical to Latin characters, they can register domains that appear identical to legitimate sites when displayed to users.

Typosquatting detection identifies domains that attempt to mimic legitimate brands by exploiting common typing mistakes. This includes keyboard layout mistakes like googel instead of google, character omissions or additions, and TLD variations like google.net instead of google.com. The system maintains a database of legitimate brand names and calculates similarity scores against known brands using Levenshtein distance and other metrics. TLS certificate analysis examines the SSL/TLS configuration of target servers, validating certificate chains, checking expiration dates, and verifying that certificates are signed by trusted CAs.

15. Performance Metrics and Evaluation

The system has been extensively evaluated to ensure it meets the accuracy and performance requirements for production deployment. The ensemble model achieves 99.70% accuracy with 99.72% precision, 99.68% recall, and 99.82% F1 score on the held-out test set. These results demonstrate that the system correctly identifies the vast majority of phishing attempts while maintaining a low false positive rate. Random Forest alone achieves 99.64% accuracy with 99.70% F1, while XGBoost achieves 99.58% accuracy with 99.62% F1, showing the ensemble improvement.

Performance benchmarks show feature extraction completes in approximately 50 milliseconds per URL, ML classification in 10 milliseconds, and the full pipeline in approximately 100 milliseconds. API response times at the 95th percentile are under 200 milliseconds, ensuring responsive user experience even under load. The system can handle concurrent requests efficiently, with throughput scaling appropriately based on available computational resources. These performance characteristics make the system suitable for real-time detection in production environments.

16. Testing and Quality Assurance

Comprehensive testing ensures the system works correctly and continues to work as changes are made. Security tests verify that the authentication, authorization, and protection mechanisms function correctly, with all five security test cases passing. Comprehensive tests verify the feature extraction pipeline, ML model predictions, API endpoints, and integration between components. Test coverage exceeds 80%, with particular focus on security-critical code paths to ensure vulnerabilities are identified and addressed promptly.

17. Deployment Guide

This section provides instructions for deploying the system in various configurations. The simplest way to use the system is through the command-line interface: run `demo.py` with the `--single` flag followed by a URL to analyze immediately, or use the `--batch` flag to analyze multiple URLs. For API deployment, use Uvicorn with the command: `uvicorn 04_inference.api:app --reload`. The API will be available at <http://localhost:8000> with interactive documentation at <http://localhost:8000/docs>. For containerized deployment, use `docker-compose up -d`.

18. Future Enhancements and Research Directions

While the current system provides comprehensive phishing detection capabilities, there are many directions for future enhancement. Short-term enhancements include completing the Tauri desktop application for production release, adding Firefox extension support for cross-browser compatibility, and developing mobile applications for iOS and Android. Long-term research directions include federated learning for privacy-preserving model training, integration with real-time threat intelligence feeds, email integration for Gmail and Outlook, SIEM integration for enterprise security platforms, and exploration of larger language models for improved AI phishing detection.

19. Conclusion

Key achievements include the development of a 93-feature extraction pipeline providing rich URL analysis, ensemble machine learning models achieving 99.70% accuracy, the innovative integration of Qwen2.5 for detecting AI-generated phishing, multiple user interfaces serving different use cases, and production-ready security features enabling safe enterprise deployment. The system is production-ready and can be deployed in various configurations to meet different requirements, whether used through the command line, integrated via the REST API, deployed as a browser extension, or run as a desktop application.

20. References and Resources

- [1] MultiPhishGuard: An LLM-based Multi-Agent System for Phishing Email Detection, arXiv, 2025.
- [2] Machine Learning Techniques for Phishing Detection: A Review, Sage Journals, 2025.
- [3] AI in Phishing Detection: A Bibliometric Review, Frontiers in AI, 2025.
- [4] PhishGuard: Leveraging NLP and ML for Email Phishing Detection, IEEE, 2025.
- [5] In-Depth Analysis of Phishing Email Detection Using ML, MDPI Applied Sciences, 2025.
- [6] Robust ML-based Detection of LLM-Generated Phishing, arXiv, 2025.
- [7] Phishing URL Detection with Neural Networks, Nature Scientific Reports, 2024.
- [8] ChatSpamDetector: Using LLMs for Phishing Email Detection, arXiv, 2024.
- [9] Digital Deception: Generative AI in Phishing, Springer Artificial Intelligence Review, 2024.
- [10] PhishTank Dataset, OpenDNS/Cisco, 2024.
- [11] OpenPhish Dataset, 2024.
- [12] UCI Machine Learning Repository - Phishing Websites Dataset.

Appendix A: Project Directory Structure

01_data/ - Raw and processed datasets 02_models/ - Trained ML models
03_training/ - Training scripts and MLflow 04_inference/ - API and service
layer 05_utils/ - Feature extraction utilities browser-extension/ - Chrome
extension gui-tauri/ - Desktop app source tests/ - Test suites viva/ -
Presentation materials

Appendix B: Technology Stack

Languages: Python 3.9+, TypeScript, Rust ML/DL: scikit-learn, XGBoost, PyTorch, Transformers Web: FastAPI, Uvicorn, React, Tauri Security: JWT, bcrypt, hashlib
Tools: Git, Docker, Conda, MLflow

Appendix C: API Quick Reference

GET / - API info GET /health - Health check POST /auth/login - Get JWT token
POST /api/v1/analyze - Analyze URL POST /api/v1/batch-analyze - Batch analyze